

Opérateurs conditionnel et Elvis dans Thymeleaf

1. Deux opérateurs à ne pas confondre

La plupart des langages de programmation proposent des opérateurs permettant de choisir une valeur selon une condition. Thymeleaf en propose deux :

Nom	Syntaxe	Principe
Opérateur conditionnel (ternaire)	condition ? valeurSiVrai : valeurSiFaux	Teste la vérité de la condition
Opérateur Elvis (expression par défaut)	valeur ?: valeurParDéfaut	Teste si la valeur est null

On nomme parfois, à tort, la première forme « opérateur Elvis ». Le nom d'Elvis est réservé à la forme abrégée `?:` (le symbole rappelle la célèbre mèche de cheveux d'Elvis Presley vue de côté). Cette distinction est importante, car les deux opérateurs ne se comportent pas de la même façon (voir la section 3).

2. L'opérateur conditionnel (ternaire)

Syntaxe

```
<p th:text="${myVariable} ? ${myValue1} : ${myValue2}"></p>
```

- Si `myVariable` est évaluée comme vraie, le code est équivalent à `<p th:text="${myValue1}"></p>`.
- Si elle est évaluée comme fausse, il est équivalent à `<p th:text="${myValue2}"></p>`.

La condition peut aussi être une comparaison ou une expression logique :

```
<span th:text="${user.age} >= 18 ? 'Majeur' : 'Mineur'">Majeur</span>
<span th:text="${user.active} and ${user.admin} ? 'Admin actif' : '-'"></span>
>
```

Les opérateurs usuels sont `==`, `!=`, `<`, `>`, `<=`, `>=` (ou `lt`, `gt`, `le`, `ge`, `eq`, `ne`), ainsi que `and`, `or` et `!` (ou `not`). Dans un attribut HTML, les alias textuels `lt` et `gt` évitent d'échapper `<` et `>`.

Quand une condition est-elle fausse ?

Dans Thymeleaf, une valeur est évaluée comme **fausse** si elle est :

- `null` ;
- le booléen `false` ;
- le nombre `0` ;

- l'une des chaînes "false", "off" ou "no".

Toute autre valeur est évaluée comme **vraie**. Attention : une chaîne vide "" et une collection vide sont donc considérées comme **vraies**.

Exemple

```
<h2 th:text="${user} ? ${user.userName} : 'Vous n'êtes pas connecté'">
  Nom d'utilisateur
</h2>
```

Remarque : le texte entre apostrophes est un littéral de chaîne. Une apostrophe à l'intérieur doit être échappée par \', ou on utilise la substitution de littéraux |...|.

Imbrication

On peut imbriquer des opérateurs conditionnels en les entourant de parenthèses :

```
<span th:text="${myVariable1} ? ${myValue1}
      : (${myVariable2} ? ${myValue21} : ${myValue22})"></span>
```

Au-delà de deux niveaux, la lisibilité chute : mieux vaut alors utiliser th:if / th:unless ou th:switch, ou calculer la valeur dans le contrôleur.

Variante sans partie « sinon »

La partie : valeurSiFaux est facultative. Si elle est omise et que la condition est fausse, le résultat est null, ce qui revient à ne rien afficher pour th:text.

```
<li th:class="${item.selected} ? 'active'">Élément</li>
```

3. L'opérateur Elvis (expression par défaut)

L'opérateur Elvis ?: est une forme abrégée du conditionnel lorsque la valeur testée est aussi la valeur retournée.

```
<!-- Forme longue -->
<p th:text="${myVariable} ? ${myVariable} : ${myValue2}"></p>

<!-- Forme courte -->
<p th:text="${myVariable} ?: ${myValue2}"></p>
```

Exemples

```
<div th:text="${errorMessage} ?: 'Aucune erreur !'"></div>

<div th:object="${user}">
  <p>Âge : <span th:text="*{age} ?: '(âge non renseigné)'">27</span>.</p>
</div>
```

Différence de comportement avec le conditionnel

L'opérateur Elvis de Thymeleaf ne remplace la valeur que si celle-ci est **null**. Le conditionnel, lui, applique les règles de véracité de la section précédente. Les deux formes ne sont donc **pas strictement équivalentes** :

myVariable vaut	<code>\${myVariable} ? \${myVariable} :</code>	<code>\${myVariable} ?:</code>
default	default	default
"Tom"	Tom	Tom
0	default	0
"no"	default	no

Dans l'exemple de l'âge, un utilisateur de 0 an s'afficherait donc bien « 0 » avec l'opérateur Elvis, alors que la forme longue afficherait « (âge non renseigné) ». Choisissez l'opérateur selon l'intention : « valeur absente » (Elvis) ou « valeur à considérer comme fausse » (conditionnel).

Le jeton « no-operation » _

Le jeton _ signifie « ne rien faire » : le contenu statique du template est conservé. C'est très utile avec les Natural Templates :

```
<span th:text="${user.name} ? : _">Visiteur</span>
```

Si `user.name` est `null`, le texte « Visiteur » écrit dans la maquette est conservé.

4. Opérateurs équivalents côté SpringEL

Dans Spring, les expressions `${...}` sont évaluées avec SpringEL, qui propose ses propres opérateurs, utilisables à l'intérieur de l'expression :

```
<!-- Ternaire SpringEL -->
<span th:text="${user != null ? user.userName : 'Anonyme'}"></span>

<!-- Elvis SpringEL -->
<span th:text="${user.userName ?: 'Anonyme'}"></span>

<!-- Navigation sûre : évite l'exception si user est null -->
<span th:text="${user?.address?.city}"></span>
```

Attention : l'opérateur Elvis de SpringEL (à l'intérieur de `${...}`) remplace la valeur si elle est `null` **ou si c'est une chaîne vide**, alors que celui de Thymeleaf (à l'extérieur de `${...}`) ne teste que `null`.

Le `?.` (*safe navigation*) retourne `null` au lieu de lever une exception lorsque l'objet est `null`. On l'associe souvent à l'opérateur Elvis :

```
<span th:text="${user?.address?.city} ?: 'Ville inconnue'"></span>
```

Résumé

- `cond ? a : b` est l'opérateur conditionnel ; `v ? : d` est l'opérateur Elvis.
- Une condition est fausse pour `null`, `false`, `0`, `"false"`, `"off"` et `"no"` ; `""` est vraie.
- L'opérateur Elvis ne teste que `null`, il n'est donc pas strictement équivalent à `v ? v : d`.
- On imbrique les conditionnels avec des parenthèses, en évitant d'aller trop loin.
- Le jeton `_` conserve le texte de la maquette ; `?.` sécurise la navigation dans les objets.

Exercices proposés

1. Afficher « Bonjour, *nom* » si un utilisateur est connecté, et « Bonjour, visiteur » sinon.
2. Que produit `<span th:text="${count} ?: 'aucun'">` quand `count` vaut `0` ? Et avec `${count} ? ${count} : 'aucun'` ? Expliquer la différence.
3. Écrire un conditionnel imbriqué qui affiche « Faible », « Moyen » ou « Élevé » selon une note, puis le réécrire avec `th:switch`.
4. Afficher la ville d'un utilisateur dont l'adresse peut être `null`, avec `?.` et une valeur par défaut.
5. Utiliser `th:classappend` avec un conditionnel pour colorer une ligne de tableau selon un statut.